

Cheap features, a small CNN or frozen UTR-LM?

An exploratory comparison for predicting 5'UTR mean ribosome load

Study repository `rewire-bio/utr-model-selection`

Author list to be confirmed by the study owner. Drafted with AI assistance; see Section 8.

Original analysis: 30 September 2026. Manuscript compiled from the archived results: October 2026.

Status of this manuscript. This is a manuscript reporting *existing* computational results. Every number comes from the companion run executed on 30 September 2026 (archived as `downloads/utr-baselines-results.zip`) or from the cited literature; values were extracted and formatted from the archived outputs, and nothing was recomputed for this manuscript. A fresh, independent reproduction is **pending**; the repository's reproduction protocol is an unapproved, unconfigured scaffold and has not been executed. No result here carries a verified or approved status. All results are exploratory: both test sets had prior exposure, and the assay is a single unreplicated measurement.

Abstract

Laboratories that measure 5'UTR libraries by polysome profiling want to predict mean ribosome load (MRL) for sequences they have not made. We compared ridge regression on cheap sequence features, a small convolutional neural network (CNN) trained from scratch, and a frozen RNA language model (UTR-LM, checkpoint `ESM2_1.4_five_species`) with three trained heads, on the 100,017 designed 5'UTRs of Sample et al. (2019), under a protocol registered before any new method was scored. On a grouped split that keeps families of related sequences (shared parent or identical 20-nt substring) on one side of the train/test boundary, the one-hot CNN had test MSE 0.560 MRL² against 0.776 for the best cheap baseline, a reduction of +0.216 [+0.176, +0.252] (95% cluster-bootstrap interval), which meets the prespecified 0.10 MRL² practical threshold. For picking the top 1% (150) of 15,004 test sequences, the CNN found 86 high-MRL sequences and the cheap baseline 83; the precision difference, +0.020 [−0.088, +0.132], was not supported, and the interval is too wide to show equivalence. Frozen UTR-LM embeddings were worse than cheap features with a ridge head on both splits, and as CNN input made no supported difference from one-hot input on the grouped split (−0.010 [−0.034, +0.016]); on a random split in which 79.3% of test records had a relative in training they lowered MSE by +0.021, below the threshold, and lowered precision. The results are exploratory: one unreplicated assay in HEK293T cells, about 70,000 training labels, one frozen checkpoint and one laptop. The endpoint is ribosome loading on an eGFP reporter, not protein output or endogenous translation. All values are existing results from a 30 September 2026 run; independent reproduction is pending.

Contents

1	Introduction	3
2	Background and related work	3
2.1	The assay	3
2.2	Models of 5'UTR translation	3
3	Materials and methods	5
3.1	Data	5
3.2	Splits	5
3.3	Methods compared	6
3.4	Metrics	6
3.5	Uncertainty and decision rule	7
3.6	Execution, mechanical checks and provenance	7

4	Results	7
4.1	The CNN reduced error; its top-1% advantage is not supported on new families . .	7
4.2	Frozen UTR-LM features gave no gain above the threshold over one-hot input . . .	9
4.3	Accuracy cost minutes and gigabytes on a shared laptop	10
4.4	Six declared examples show where the errors sit	10
4.5	Errors concentrate at low read depth and in one trajectory family	11
4.6	Variant effects, split design and device choice	11
4.7	Reproduction status	12
5	Discussion	12
5.1	The choice depends on what will be done with the predictions	12
5.2	What would change these conclusions	13
6	Limitations	13
7	Data, code and licence availability	14
8	AI assistance, review and disclosure	15
A	Reproducing the historical run	17
B	The grouped split	17
C	Supplementary tables	18
D	Selection flowchart	24
E	Provenance, supplement and evidence mapping	25

1 Introduction

A laboratory with a 5'UTR reporter library and measured mean ribosome load (MRL, the average number of ribosomes on each mRNA molecule) often wants to predict MRL for sequences it has not made yet, either to estimate values or to choose a small set of candidates for follow-up. We consider three options: regression on cheap, interpretable sequence features; a small convolutional network trained on the library's own labels, in the style of Optimus 5-Prime [1]; and a pretrained RNA language model whose representation is fed to a trained head [2, 3]. The last option adds an embedding step and a dependency on pretrained weights; the question is whether it improves accuracy here.

We compared these options on the designed library of Sample et al. [1], processed by mRNABench [3], under a protocol fixed before any new method was scored on a test set. Two questions are distinguished: whether a method predicts MRL values more accurately (mean squared error), and whether it puts more high-MRL sequences into a fixed top-1% pick list (precision at 1% capacity). Two splits are also distinguished, because related sequences in this library otherwise straddle the train/test boundary. The contributions are (i) a paired, cluster-bootstrapped comparison of ten methods on a random and a grouped split with prespecified practical thresholds; (ii) a separation of accuracy gains from pick-list gains; (iii) a documented accelerator divergence for the language-model embeddings; and (iv) a runnable companion and results archive behind every number. This comparison is between workflows, not matched architectures, and every result is exploratory.

2 Background and related work

2.1 The assay

In the Sample et al. assay, in vitro transcribed (IVT) reporter mRNAs are transfected into cells, the lysate is separated on a sucrose gradient into fractions carrying different numbers of ribosomes, and each fraction is sequenced. For each 5'UTR, MRL is the sum over fractions of its relative read count in each fraction multiplied by that fraction's ribosome number [1] (Figure 1). MRL leaves out the ribosome-free mRNA fraction, so a high-MRL sequence may still have a large untranslated fraction [4].

2.2 Models of 5'UTR translation

Sample et al. trained a CNN (Optimus 5-Prime) on MPRA data and reported variant effects as squared Pearson correlations of log changes [1]. UTR-LM [2, 5] pretrains an ESM2-style model on 5'UTRs and, by default, fine-tunes it on downstream labels; in the authors' ablation on a random 50-nt library with a rank split, a frozen model with a trained MLP head reached Spearman 0.6 against 0.962 fully fine-tuned [6]. mRNABench [3, 7] benchmarks linear probes on frozen embeddings from many RNA models, and showed that probes and the k -mer baseline lost much of their random-split Pearson correlation when one Kozak/uAUG combination was held out (Figure 2). In its appendix table the k -mer baseline lost 0.34 in Pearson correlation and a probe on `utr1m-mr1` lost 0.33, while another UTR-LM variant, `utr1m-te-e1`, lost 0.17. The figure shows one point per model family, and its `utr1m` point (about 0.56 to 0.39) matches `utr1m-te-e1`. Karollus et al. [8] found that MPRA-trained MRL models correlate only weakly with endogenous translation measurements.

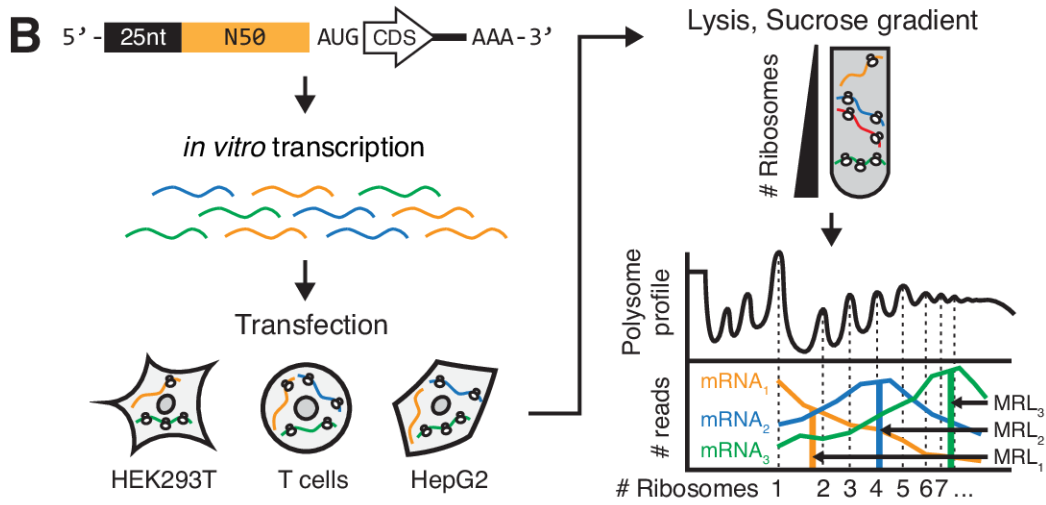


Figure 1: How a polysome-profiling reporter assay turns reads across ribosome fractions into MRL. Reporter mRNAs with a fixed 25-nt segment and a variable 50-nt 5'UTR are transfected into cells, separated on a sucrose gradient, and read counts across ribosome fractions give MRL. This panel shows a 2024 library with the same 25 nt + 50 nt construct, not the 2019 data used here. Adapted (cropped) from Castillo-Hair et al. 2024, Fig. 1b [4], CC BY 4.0.

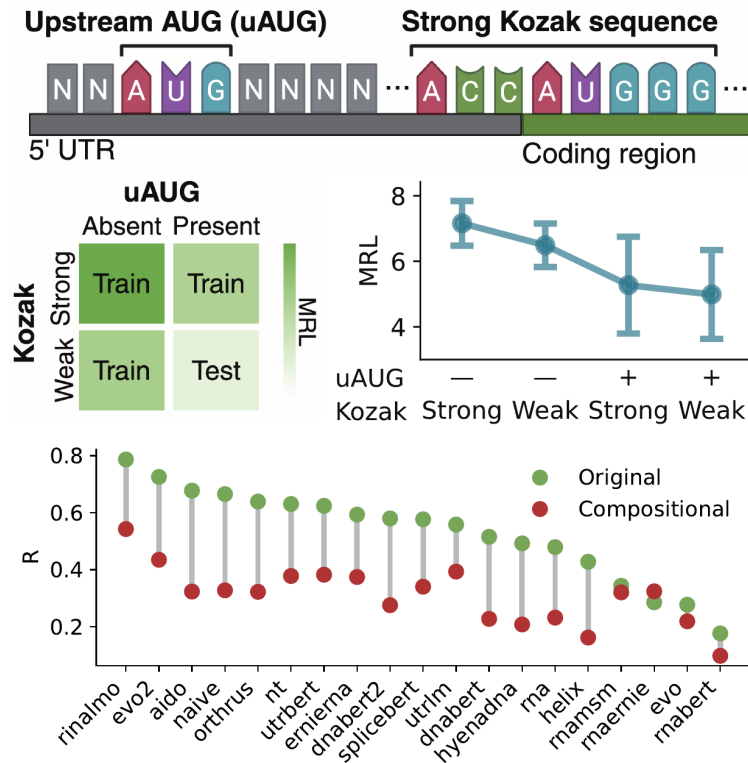


Figure 2: Random versus compositional split in mRNABench. Linear probes on frozen embeddings, and the k -mer baseline, lost much of their random-split Pearson r when one Kozak/uAUG combination was held out; the supervised CNN is not shown. From Shi et al. 2025 (mRNABench), Fig. 7 [3], CC BY 4.0; cropped.

3 Materials and methods

The protocol (`utr-mrl-designed-choice-v1`, `companion/PROTOCOL.md`) was registered on 30 September 2026 at 08:56:32Z, amended once at 09:00:14Z before any test scoring (example rule only), and followed by nine dated reporting clarifications written after scoring; none changes a method, prediction, threshold or verdict. Its registered and amended texts are archived with their SHA-256 digests.

3.1 Data

Each designed-library record is a 25-nt defined leader, a 50-nt insert and the eGFP coding sequence, measured in HEK293T cells 12 hours after transfection (GEO GSM3130443 [9]). The inserts include 50-nt fragments of 35,212 human 5'UTRs, 3,577 ClinVar variants, genetic-algorithm designs and step-wise evolution trajectories in which every intermediate was synthesised (80 trajectories, 7,526 UTRs). The `mcherry` sub-library label records design origin: the whole designed library was cloned into the same eGFP vector. Two of the 13 labels, `sergii_utrs` and `egfp_poor_performers`, have no documented definition. The library was measured once, with no duplicate (GEO GSE114002), so there is no replicate-based ceiling on achievable accuracy; the sources do not state its RNA chemistry.

Processed records come from the mRNA-Bench Hugging Face dataset [10] (file `mrl-sample-designed.parquet`, revision `ef67f7c`); metadata (sub-library, parent `mother` sequence, read count) come from GEO and are joined one to one on the insert, with the GEO MRL required to equal the processed target exactly for all 100,017 records (maximum difference 0.0). Only the insert varies: the other 805 nt of each 855-nt processed record are identical in all records. A replay of an earlier full-sequence composition control from rewire-benchmarks [11] therefore gives near-identical predictions to insert-only composition (maximum difference below 0.001 MRL; test MSE 1.914 on the random split). Every method below uses the insert alone. Appendix Table 10 gives the sub-library counts.

3.2 Splits

Random split. The random split (called `historical` in the code) is mRNA-Bench's seed-2541 random split (70,011 / 15,003 / 15,003 records), reconstructed from source row order and checked against its published SHA-256. Its test set had already been used for two published controls, so results on it are not a fresh test. It also mixes relatives, such as trajectory intermediates and ClinVar variants, with their source sequences. Linking records that share a GEO parent sequence or any identical 20-nt substring gives 33,575 families. In the random split, 79.3% of test records had a relative in training, and training labels had a within-family intraclass correlation of 0.495.

Grouped split. The grouped split assigns whole families to training, validation or test (69,996 / 15,017 / 15,004 records). It was registered before scoring, but whole-dataset label summaries by sub-library had been viewed, so it is not an untouched cohort either. Families holding more than 1% of records go to training, and one does: 14,101 records (14.1%), 98% genetic-algorithm designs, mean MRL 6.59 against 5.74 overall. The remaining families are shuffled (seed 20260930) and assigned to test until it reaches 15% of records, then to validation (Appendix B). The grouped test set therefore has no `step_random_to_best_no_uatgs` records, fewer other genetic-algorithm designs, and a high-MRL rate of 0.055 against 0.095. It is a defined transfer test to sequences with no shared parent or 20-nt substring in training, not a leakage-free estimate for every designed sequence. Methods should be compared within a split; the two test sets differ in overlap, composition and base rate.

3.3 Methods compared

Cheap ridge baselines. Ridge regression on nucleotide composition (A/C/G/T and GC fractions); counts of all 1- to 3-mers (84 features); position-specific one-hot (200 features); annotation features computed from the insert and the known reporter context (upstream AUGs [uAUGs] and their frame, uAUG-stop pairs, stop codons, Kozak context); and `cheap_combined`, which concatenates the last three. Features are standardised on training data and the penalty is chosen from $\{10^{-3}, \dots, 10^4\}$ by validation MSE. The reference is the cheap model with the lowest validation MSE: `cheap_combined` on both splits.

Supervised CNN. An Optimus 5-Prime-style network (three convolutional layers of 120 width-8 filters, a 40-unit dense layer, dropout 0.2) trained from scratch on one-hot inserts with Adam (learning rate 10^{-3} , batch 128, at most 20 epochs, early stopping after 3 epochs without validation improvement). Seeds 0, 1 and 2 were trained; headline numbers use seed 0, fixed in advance.

Frozen UTR-LM. The `ESM2_1.4_five_species` checkpoint from the UTR-LM repository at b77b589 [5] (6 layers, 128 dimensions, 1,208,043 parameters). Its final-layer insert embeddings fed three heads: ridge on their mean, ridge on all 50 positions (6,400 features), and a CNN head with the one-hot CNN's layer design. The CNNs take 4 and 128 input channels, so their parameter counts differ: this compares workflows, not matched architectures. "Frozen" means the language model's weights stay fixed and only the head learns; "fine-tuned", the UTR-LM authors' default, means those weights are also updated. Nothing here was fine-tuned.

The case for this checkpoint is documentary. The UTR-LM paper [2] describes a baseline pretrained only by masked-nucleotide prediction on unlabelled Ensembl 5'UTRs from five species, and `ESM2_1.4_five_species` is the only pretrained file whose name carries no token for the Cao et al. endogenous data, a downstream library, structure or free energy. It also loads strictly into plain ESM2. No text names the file and no sequence-overlap audit was run; human 5'UTR fragments in this library may appear, unlabelled, in that corpus. The `FS4.x` checkpoints and MultiMolecule `utr1m-mr1` [12] (used by mRNA Bench) were excluded because Sample et al. random-library sequences were in their pretraining; the sources do not show that they saw MRL labels. The chosen model beat the commonest base at masked-nucleotide prediction (0.416 against 0.329), which confirms it loaded and runs, nothing more. Embeddings were computed on CPU (Section 4.6). Hyperparameters for every method were chosen on validation data, and each method was scored on the test set once.

Orthrus [13] was considered and excluded: its pinned `mamba-ssm==1.2.0.post1` dependency [14] failed to build without CUDA, and it was trained on full mature RNAs. That says nothing about its accuracy. The failed installation logs are archived.

3.4 Metrics

MSE is the mean squared difference between predicted and measured MRL, in MRL^2 ; lower is better. Predicting the training mean gives 2.224 on the grouped test set. $\mathbf{R}^2$ is the coefficient of determination, $1 - \text{SSE}/\text{SST}$ with SST about the test mean; zero matches predicting the test mean. It is not the squared Pearson correlation that Sample et al. report as r^2 .

Precision at 1% capacity matches a follow-up budget. A record is high-MRL if its measured MRL is at or above the 90th percentile of that split's training labels (7.402 on the grouped split, 7.360 on the random split). Precision is the high-MRL fraction of the $k = 150$ top-ranked test records (1%), with ties broken by lower source index as a fixed convention. Chance is the test high-MRL rate: 0.055 on the grouped split (about 8 of 150) and 0.095 on the random split (about 14). For the constant training-mean predictor every record ties, so its observed precision reflects the tie rule, not chance.

3.5 Uncertainty and decision rule

Intervals are 95% percentile intervals from 2,000 cluster-bootstrap resamples of whole test families (seed 7) [15], describing sampling uncertainty for a fixed model. Paired differences against the reference are computed within each draw. In each draw the capacity is 1% of the resampled list: 140 to 160 records on the grouped split and 124 to 318 on the random split, where large families can repeat. Training-seed variability is reported separately (Appendix Table 7). Neither captures the variability of repeating the assay.

The protocol declares that a method *adds practical value* if it lowers MSE by at least 0.10 MRL², or raises precision at 150 by at least 0.05, against the reference, with the 95% interval excluding zero. A smaller supported difference is *below the practical threshold*; an interval spanning zero is *no supported difference*. The companion’s verdict function additionally labels an interval wholly below zero *worse than reference*. An interval ending exactly at zero (+0.000) does not exclude zero. The same rule compares the UTR-LM methods against the one-hot CNN. The thresholds are this comparison’s prespecified decision criteria, not validated measures of biological benefit, and “no supported difference” does not mean equivalence. None of the 22 registered comparisons is adjusted for multiplicity.

3.6 Execution, mechanical checks and provenance

Every command was run on 30 September 2026 on an Apple M4 laptop (10 cores, 16 GB RAM, macOS 26.6.2, Python 3.11.13, torch 2.4.1). The companion fetches 15 files (32.1 MB) and checks each against a pinned SHA-256. An independent script, `verify.py`, which shares no code with the main script, rebuilt the 33,575 families by a separate search and confirmed that none spans two grouped splits, recomputed every test metric from saved predictions (maximum difference 0), refitted the composition baseline with scikit-learn (differences about 1.4×10^{-7}) and reproduced the published composition MSE (1.914439715839851) (Appendix Table 13). It does not retrain the CNNs. A clean-environment rerun from the code archive on the same machine repeated preparation, the UTR-LM embedding, all 18 deterministic fits and the grouped seed-0 CNN on MPS; every test prediction matched the original exactly, including the CNN (test MSE 0.559972 both times; Appendix Table 14). Other machines, operating systems and torch versions were not tested, so no cross-platform tolerance is established. Further mechanical checks (metric recomputation for all 11 methods on both splits, bootstrap-depth reconstruction and a CPU-only smoke run) were run by a separate automated orchestration session (Codex). These records are historical; none was re-executed for this manuscript. The tables below are formatted from the digest-checked archived results; values that also appear in the original article are cross-checked against it (Appendix E).

4 Results

4.1 The CNN reduced error; its top-1% advantage is not supported on new families

Table 1 lists every method on both splits and Table 2 the paired differences behind the verdicts; Figure 3 plots the MSE differences.

On the grouped split the whole interval for the CNN’s MSE reduction lies above the 0.10 threshold, and its test MSE across three seeds (0.559 to 0.570) does not change that verdict. For the top 150 picks the result differs. The CNN’s first 150 grouped-test picks held 86 high-MRL sequences against 83 for the cheap baseline, about 10 times the chance rate for both. The gain of +0.020 [−0.088, +0.132] is not supported: this evidence cannot exclude a CNN advantage of around 0.1 in precision, or a comparable disadvantage. The CNN’s own precision ranged from 0.533 to 0.580 across seeds, wider than the observed difference. On the random split the CNN put

Table 1: Test results for 15,003 (random) and 15,004 (grouped) records. MSE in MRL^2 with 95% cluster-bootstrap intervals beneath each estimate; P@150 is precision at 150 (1% capacity). CNN times are seed 0; ridge times include feature computation and the alpha search; the last two columns give the random-split value above the grouped-split value; UTR-LM rows exclude the 134.8s embedding pass. Training-mean precision reflects the tie rule, not chance. Inputs and adaptation: training mean, none; composition, 1–3-mers and positional one-hot, insert with ridge; uAUG/Kozak and cheap combined, insert plus reporter context with ridge; CNN, insert, trained from scratch; UTR-LM + ridge heads, insert, frozen embedding with ridge; UTR-LM + CNN head, insert, frozen embedding with a trained CNN. Source: archived `run/metrics.json` and `run/failures.json`.

Method	Random split			Grouped split			Fit time, s	Peak mem., GB
	MSE	R ²	P@150	MSE	R ²	P@150	rand.	rand.
	[95% CI]		[95% CI]	[95% CI]		[95% CI]	grp.	grp.
Training mean	2.366 [2.234, 2.481]	0.000	0.053 [0.008, 0.094]	2.224 [2.096, 2.384]	−0.022	0.040 [0.013, 0.077]	0.0 0.0	0.22 0.22
Composition	1.914 [1.834, 2.004]	0.190	0.327 [0.172, 0.401]	1.888 [1.807, 1.975]	0.132	0.233 [0.143, 0.327]	0.2 0.1	0.36 0.35
1–3-mer counts	1.225 [1.177, 1.267]	0.482	0.533 [0.427, 0.600]	1.162 [1.111, 1.210]	0.466	0.540 [0.444, 0.636]	13.2 13.0	0.27 0.28
Positional one-hot	1.769 [1.709, 1.839]	0.252	0.360 [0.154, 0.532]	1.767 [1.702, 1.836]	0.188	0.360 [0.234, 0.452]	1.1 0.4	0.37 0.37
uAUG/Kozak features	1.280 [1.228, 1.326]	0.459	0.340 [0.258, 0.417]	1.193 [1.138, 1.248]	0.452	0.307 [0.220, 0.380]	1.4 0.7	0.30 0.30
Cheap combined (reference)	0.849 [0.808, 0.888]	0.641	0.580 [0.386, 0.689]	0.776 [0.739, 0.814]	0.643	0.553 [0.456, 0.651]	14.6 20.7	0.55 0.45
CNN, one-hot (seed 0)	0.427 [0.389, 0.465]	0.819	0.787 [0.617, 0.866]	0.560 [0.529, 0.595]	0.743	0.573 [0.470, 0.669]	205 250	1.32 1.18
UTR-LM mean + ridge	1.184 [1.143, 1.222]	0.499	0.460 [0.312, 0.547]	1.139 [1.093, 1.183]	0.477	0.347 [0.242, 0.466]	0.6 0.2	0.30 0.29
UTR-LM per-position + ridge	0.939 [0.904, 0.981]	0.603	0.513 [0.367, 0.614]	1.020 [0.981, 1.061]	0.531	0.380 [0.307, 0.480]	106.1 89.5	3.22 2.75
UTR-LM + CNN head (seed 0)	0.407 [0.376, 0.440]	0.828	0.660 [0.512, 0.771]	0.570 [0.539, 0.602]	0.738	0.567 [0.487, 0.680]	269 263	2.18 2.19

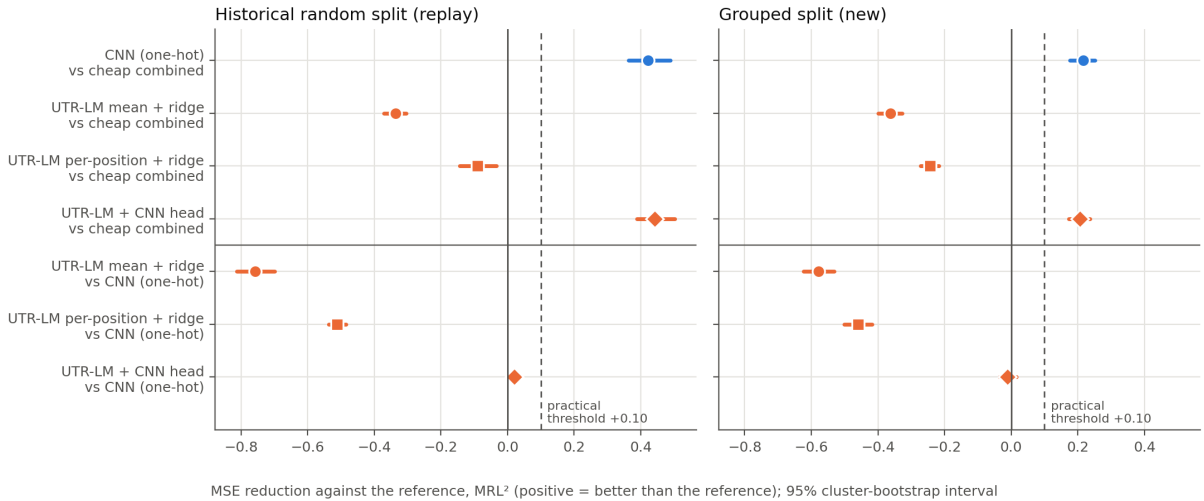


Figure 3: MSE reduction against cheap combined (upper rows) and against the one-hot CNN (lower rows), with 95% cluster-bootstrap intervals and the 0.10 MRL^2 threshold (dashed). Positive values favour the named method. Source: original figure `article/assets/fig-paired-differences.png`, generated by the companion’s `make_figures.py` from `run/metrics.json` (30 September 2026). (“historical” in the figure panels and code is the random split.)

Table 2: Paired differences and prespecified verdicts. Positive favours the first-named method. Verdicts follow the thresholds of 0.10 MRL² and 0.05 precision; “worse” abbreviates “worse than reference”; an interval ending exactly at zero (+0.000) does not exclude zero. All 22 registered comparisons are in Appendix Table 5. Source: archived `run/metrics.json, comparisons`.

Comparison	Split	MSE reduction [95% CI]	Verdict	P@150 gain [95% CI]	Verdict
CNN vs cheap combined	random	+0.422 [+0.363, +0.488]	adds practical value	+0.207 [+0.124, +0.292]	adds practical value
CNN vs cheap combined	grouped	+0.216 [+0.176, +0.252]	adds practical value	+0.020 [−0.088, +0.132]	no supported difference
UTR-LM + CNN vs cheap combined	random	+0.443 [+0.388, +0.502]	adds practical value	+0.080 [+0.000, +0.208]	no supported difference
UTR-LM + CNN vs cheap combined	grouped	+0.206 [+0.173, +0.237]	adds practical value	+0.013 [−0.087, +0.149]	no supported difference
UTR-LM per-position + ridge vs cheap combined	random	−0.090 [−0.144, −0.033]	worse	−0.067 [−0.141, +0.056]	no supported difference
UTR-LM per-position + ridge vs cheap combined	grouped	−0.244 [−0.272, −0.216]	worse	−0.173 [−0.278, −0.047]	worse
UTR-LM mean + ridge vs cheap combined	random	−0.335 [−0.371, −0.305]	worse	−0.120 [−0.203, −0.015]	worse
UTR-LM mean + ridge vs cheap combined	grouped	−0.363 [−0.400, −0.327]	worse	−0.207 [−0.335, −0.073]	worse
UTR-LM + CNN vs CNN	random	+0.021 [+0.009, +0.034]	below practical threshold	−0.127 [−0.179, −0.023]	worse
UTR-LM + CNN vs CNN	grouped	−0.010 [−0.034, +0.016]	no supported difference	−0.007 [−0.103, +0.122]	no supported difference

118 high-MRL sequences in its first 150 against 87, a gain of +0.207 [+0.124, +0.292], the only supported precision gain over the reference in either split.

Among the cheap baselines, every single feature set was worse than `cheap_combined` on MSE on both splits; on precision, 1–3-mer counts alone showed no supported difference from it (Appendix Table 5).

4.2 Frozen UTR-LM features gave no gain above the threshold over one-hot input

Ridge on UTR-LM embeddings was worse than ridge on cheap features on both splits, averaged or per position, and worse than the one-hot CNN. As input to a CNN head, UTR-LM embeddings showed no supported difference from one-hot input on the grouped split (MSE 0.570 against 0.560; −0.010 [−0.034, +0.016]). On the random split they were slightly better on MSE (+0.021 [+0.009, +0.034], below the threshold) and worse on precision (−0.127 [−0.179, −0.023]). On the grouped split, validation and test MSE ordered the two heads differently; neither ordering is supported. Across seeds the UTR-LM CNN head’s grouped test MSE ranged from 0.538 to 0.570 (Appendix Table 7).

This says nothing about fine-tuning; in the authors’ own ablation, under a different library and split, fine-tuning outperformed a frozen model with an MLP head (Section 2) [6]. mRNA-Bench’s saved results for this dataset [7] show the same ordering under a different protocol: on its random split, a supervised CNN reached Pearson r 0.862 (five seeds), a k -mer and GC ridge probe 0.788, a probe on the excluded `utr1m-mr1` 0.649 and one on RiNALMo, another RNA language model, 0.821. On a harder k -mer cluster split the k -mer and `utr1m-mr1` probes fell to 0.729 and 0.619. The probes used the full 855-nt sequence; the CNN’s input cannot be confirmed from the pinned code. This is context, not a replication or a verdict on RNA foundation models.

4.3 Accuracy cost minutes and gigabytes on a shared laptop

The whole run took 3,417 s (56.9 minutes). The laptop was shared and swapping (about 7 GB) during the grouped fits, so these are single-run receipts, not a benchmark (Figure 4; per-step times in Appendix Table 12). Cheap combined ridge took 15 to 21 s, mostly computing k -mer features, with the alpha search included, using under 0.6 GB. One CNN seed took 174 to 313 s on the Apple GPU (Metal Performance Shaders, MPS). The plotted seed-0 one-hot CNN took 205 s (random) and 250 s (grouped), with peak memory of 1.2 to 1.3 GB. All three seeds together took 588 s and 733 s for the one-hot CNN, and 876 s and 777 s for the UTR-LM CNN head (random, then grouped). On CPU (4 threads) one epoch took 151 s against 11 s on MPS in a pre-registration probe, so no full CPU-only run was attempted. UTR-LM added a 4.87 MB checkpoint, a 134.8 s CPU embedding pass (1.76 GB peak), a 1.28 GB embedding cache, and the run's largest memory step, 3.2 GB for per-position ridge.

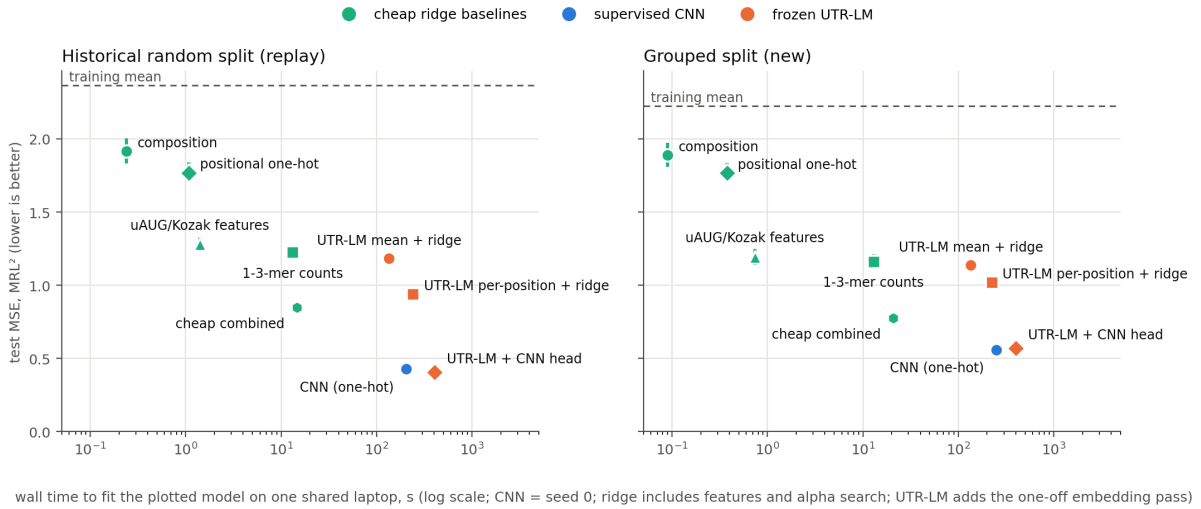


Figure 4: Test MSE (lower is better) against wall time to fit the plotted model, log scale, random and grouped splits side by side, with 95% intervals. CNN points use the seed-0 time, ridge includes feature computation and the alpha search, and UTR-LM adds the embedding pass. Single-run times on a shared laptop. Source: original figure [article/assets/fig-accuracy-vs-cost.png](#) (companion run, 30 September 2026). (“historical” in the figure panels and code is the random split.)

4.4 Six declared examples show where the errors sit

The protocol fixed the example rule before test scoring: grouped-test records whose measured MRL is closest to the 10th, 50th and 90th percentiles; the record with the largest seed-0 CNN error; and the lowest-index SNV variant whose reference is also in the test set (Table 3). Every record shares one reporter context: the 25-nt leader, the record's own 50-nt insert and the eGFP coding sequence.

The 10th-percentile insert has an out-of-frame ATG at position 32; both CNN heads came within about 0.5 of the measured 3.28, while both ridge models predicted above 5. The 90th-percentile insert has in-frame upstream ATGs at positions 42 and 45. Every method underpredicted it; UTR-LM + CNN came closest (7.07 against 7.22). The largest CNN error is a human UTR fragment measured at 1.99 and predicted at 6.70 to 8.10 by every method. It has no upstream ATG, and ACC (a strong Kozak context) precedes the main AUG, so it carries none of the upstream-ATG or weak-Kozak features known to lower loading. It had 328 reads, against a test median of 616. One record cannot show whether the low value reflects biology the models miss or measurement noise. The SNV pair differs by one C>T change at insert position 24. Measured MRL rose by 0.66 (0.67 from the rounded values in Table 3); the predicted changes were +0.17

Table 3: Declared examples from the grouped test set, with predictions from four methods (MRL). “Index” is the source index; the 50-nt insert is printed beneath each row; `egfp_poor_performers` has no documented definition. These six inserts are the only variable-region source sequences embedded in the results archive, which also holds the constant 780-nt tail (Section 7). Source: archived `run/metrics.json`, `splits.grouped.examples`.

Rule	Sub-library	Index	Reads	Measured MRL	Cheap comb.	CNN	UTR-LM + CNN	UTR-LM pos. ridge
10th percentile	<code>egfp_poor_performers</code>	1422	4,080	3.28	5.15	3.80	3.76	5.12
insert:	GGCTACCTCTCAATAAGTTGTAGTAGCAAACATGTCCAGAGTTGTTTGCA							
50th percentile	<code>human_utrs</code>	60165	890	5.92	6.32	6.06	6.01	6.51
insert:	CGCGCAGCGCGCGCAGAGCGTCTCCAGGAAGGAAGCAGCCACCCTCGCC							
90th percentile	<code>target_allow_uaug_allow_stop</code>	53663	1,326	7.22	6.37	6.72	7.07	5.17
insert:	TTGTCAAATTTTATTTTAGTCAGTGTTATAGTAGTAGTAAATGATGAAA							
Largest CNN error	<code>human_utrs</code>	38420	328	1.99	7.17	6.92	6.70	8.10
insert:	CTTTCTCGTGCTCAGTCACATCTTTCTCTTCTCCACCCCGAGGGACC							
SNV variant	<code>snv</code>	20	12,256	5.49	5.59	5.62	5.93	5.80
insert:	CCCACCCGGGCTCTCTCCTGGCTTCCACCCCGCGCCGGCTTCCACC							
SNV reference	<code>snv</code>	0	22,646	4.82	5.42	5.55	5.99	5.48
insert:	CCCACCCGGGCTCTCTCCTGGCTTCCACCCCGCGCCGGCTTCCACC							

(cheap), +0.07 (CNN), −0.06 (UTR-LM + CNN) and +0.32 (UTR-LM ridge). Every method missed at least half of the change, and one got its sign wrong.

4.5 Errors concentrate at low read depth and in one trajectory family

Low read depth. Grouped-split CNN MSE was 0.785 in the lowest read-depth quartile (100 to 315 reads) and 0.419 in the highest; for the cheap baseline, 1.024 and 0.606 (Figure 5, right; Appendix Table 8). The 289 records with CNN errors above 2 MRL had a median of 485 reads against 616 overall. For their random library, Sample et al. note that low-read UTRs give noisier MRL; with no replicate here, noise cannot be separated from model error. These are point estimates without intervals.

One trajectory family. In `step_worst_to_best_allow_uatg`, the only sub-library where the grouped-split CNN’s point estimate was worse than the cheap baseline’s, its MSE was 1.134 against 0.405 (Figure 5, left; Appendix Table 9). That sub-library has 210 grouped-test records in 5 families, the largest with 101 records. But 82% of the CNN’s squared error in that sub-library came from a different family of 49 records (45 of them containing an ATG), where CNN MSE was 3.98 and the cheap baseline’s 0.59. On the random split the same sub-library, 744 test records in 38 families, showed the opposite (CNN 0.314, reference 0.723). This is one family, not evidence that the CNN fails on uAUG designs, but an aggregate MSE can hide it. Check error by family.

4.6 Variant effects, split design and device choice

Variant effects. No method recovered the one SNV change in Table 3, and this run does not test variant-effect prediction. The original study reported r^2 0.555 across 1,597 SNV pairs: a squared Pearson correlation of log2 changes on high-read records, from a retrained model that had seen designed-library sequences [1].

Split design. Between the splits, the CNN’s MSE advantage fell from +0.422 to +0.216 MRL² and its precision advantage lost support; because overlap, composition and base rate all change, the drop cannot be attributed to leakage alone.

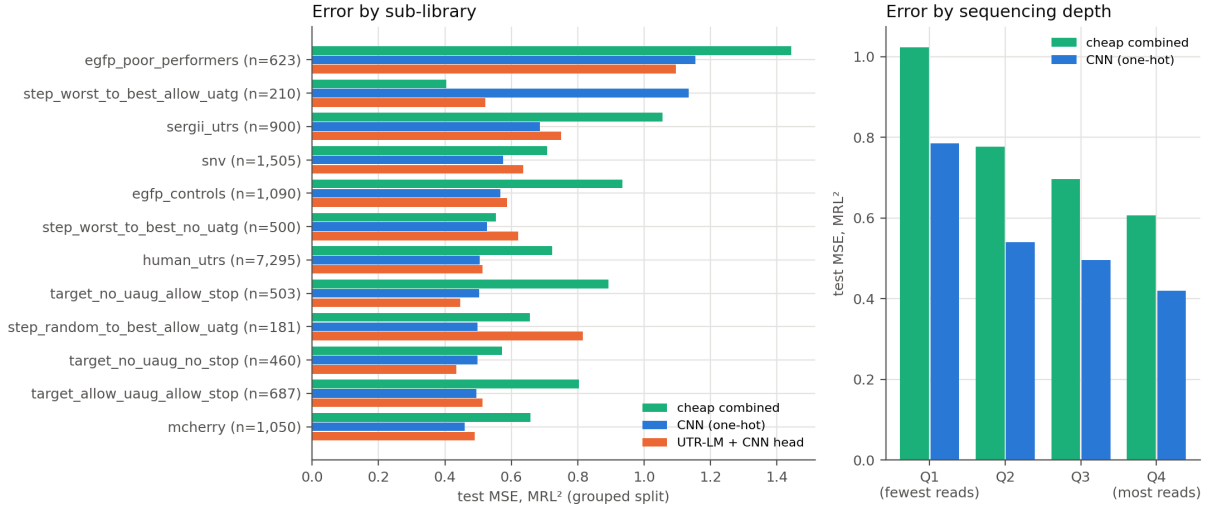


Figure 5: Grouped-split test MSE by sub-library (left) and by read-depth quartile (right). Point estimates without intervals; some sub-libraries have only five test families. Source: original figure [article/assets/fig-error-by-library-and-depth.png](#) (companion run, 30 September 2026). (“historical” in the figure panels and code is the random split.)

Device divergence. On this machine with torch 2.4.1, UTR-LM’s final-layer output on Apple MPS differed from CPU (maximum absolute difference 7.6 on 64 inserts), and masked-nucleotide accuracy on the first 1,000 inserts of the data file fell from 0.412 to 0.335, about the share of the commonest base, with no error raised. All embeddings were therefore computed on CPU; the cause was not found, and PyTorch’s reproducibility notes [16] do not promise CPU/GPU agreement. The CNNs trained on MPS after their untrained outputs matched CPU within 1.1×10^{-7} , and after training their validation MSE on MPS and CPU agreed to within about 3×10^{-8} for all 12 fits (Appendix Table 11). That is one metric on one setup, not general MPS parity; test predictions were made on CPU. Users should compare CPU and accelerator outputs on their own setup before trusting them.

4.7 Reproduction status

Not yet reproduced under this repository’s harness. The values above are imported from the 30 September 2026 companion run; a same-machine clean-environment rerun repeated a subset (Appendix Table 14). No independent reproduction on other hardware has been performed for this manuscript, and the repository’s reproduction protocol has not been designed, approved or executed.

5 Discussion

5.1 The choice depends on what will be done with the predictions

Table 4 and Appendix Figure 7 summarise the choice. For new families where MRL values are needed, the grouped split supports the one-hot CNN. For a top-1% pick list, it shows no supported difference between the CNN and cheap combined ridge, so ridge is the cheaper choice, though the interval does not rule out a CNN gain of around 0.1. For relatives of the training sequences, such as further steps of a design trajectory, the random-split evidence applies and the CNN was better on both metrics. Frozen UTR-LM input gave no gain above the prespecified threshold over one-hot input to the same CNN, so on this evidence its extra embedding step is not justified. No supported difference is a finding, not a missing result, and not a demonstration of equivalence.

Table 4: Decision summary. P@150 is precision at 150. Grouped-split numbers unless stated; 95% cluster-bootstrap intervals; all results exploratory and measured with about 70,000 training labels.

Situation	What the evidence supports	Key numbers	Main caveat	Evidence
New sequence families; MRL values needed	One-hot CNN	Grouped MSE 0.560 vs 0.776; reduction +0.216 [+0.176, +0.252]	Seed-0 model; one 49-record family failed badly	Tables 1, 2; Fig. 5
New sequence families; testing the top 1%	No supported difference between the CNN and cheap combined ridge; ridge is cheaper	P@150 0.573 vs 0.553; gain +0.020 [-0.088, +0.132]	Interval allows a CNN gain of about 0.1 or a similar loss; CNN seeds spanned 0.533–0.580	Table 2
Candidates are close relatives of training sequences	One-hot CNN on both metrics	Random split: MSE +0.422 [+0.363, +0.488]; precision +0.207 [+0.124, +0.292]	79.3% of that test set has a relative in training; different test population	Table 2
Considering frozen UTR-LM features	Not with a ridge head; with a CNN head, no supported difference from one-hot	Per-position ridge vs cheap -0.244 [-0.272, -0.216]; UTR-LM + CNN vs CNN -0.010 [-0.034, +0.016]	One checkpoint, frozen only	Table 2; Fig. 3
Only a CPU and seconds of fitting	Cheap combined ridge	20.7 s including alpha search; 0.45 GB peak memory	Higher MSE than the CNN; times from a shared laptop	Table 1; Fig. 4
Single-variant effects needed	Nothing tested here	One declared SNV pair: measured +0.66, predicted -0.06 to +0.32	One pair is an example, not an evaluation	Table 3
Assay, cells, chemistry or label count differ	Outside the tested scope	–	Run a grouped comparison on your own labels	Section 6

5.2 What would change these conclusions

A fresh cohort of new designs in the same assay would turn these exploratory numbers into a test, and a replicated measurement would show how much of the remaining error is noise. A CNN precision gain at 150 with an interval above zero would move the pick-list recommendation to the CNN. A learning curve at a few thousand labels would show whether the ordering holds for small libraries. A fine-tuned UTR-LM, or another pretrained model, that beat the one-hot CNN by more than 0.10 MRL² on a grouped split would reopen the case for foundation-model features on this assay.

6 Limitations

- **Protein output or therapeutic effect.** MRL leaves out the ribosome-free mRNA fraction, so a high-MRL sequence may still have a large untranslated fraction [4].
- **Endogenous mRNAs.** MPRA-trained MRL models correlated only 0.11 to 0.25 with six endogenous translation measurements in the main comparison of Karollus et al. [8] (Figure 6; their Figure 7 splits it by UTR length and training set).
- **Other reporters, cell lines or chemistries.** One eGFP reporter in HEK293T, one unreplicated run, chemistry not stated.
- **Few labels.** Every model trained on about 70,000 labels. No learning curve was measured, so nothing here says a frozen model would do better with few labels.
- **Variable-length UTRs.** Every insert is 50 nt, and the CNN takes a fixed 50-nt input.
- **Fine-tuned, larger or other foundation models.** One small frozen checkpoint was tested; Orthrus was excluded for installation and scope reasons, not accuracy.

- **Confirmatory claims.** Both test sets had prior exposure (the random test set was used for two published controls; the grouped split was registered after viewing whole-dataset label summaries); every result is exploratory, and the 22 comparisons are unadjusted.
- **Uncertainty scope.** Cluster-bootstrap intervals describe sampling uncertainty for fixed models; training seeds are reported separately and assay repeatability is not captured. Per-sub-library and read-depth analyses are point estimates.
- **Checkpoint provenance.** No sequence-overlap audit was run between the library and the UTR-LM pretraining corpus.
- **Platform.** All times and the exact-reproduction check come from one shared Apple M4 laptop; CNN training used MPS; no cross-platform tolerance is established, and seeded CPU-only training was not tested for exact repetition.
- **Review.** Drafting, editorial review and fact checks of the original article were automated (Claude Opus); the independent mechanical checks were run by another automated session (Codex). No human scientific review has taken place.

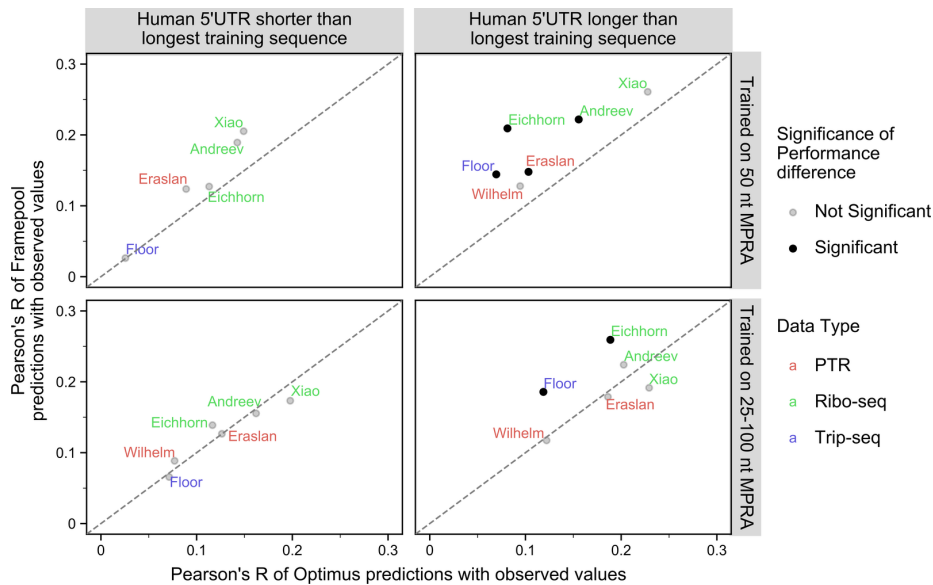


Figure 6: MPRA-trained MRL predictions transfer poorly to endogenous translation. Correlations of FramePool against Optimus predictions with six endogenous translation datasets, all below about 0.26. From Karollus et al. 2021, Fig. 4 [8], CC BY 4.0.

7 Data, code and licence availability

The study repository (`rewire-bio/utr-model-selection`) contains this manuscript and its build (`make paper-imported`); the original article (`article/original.md` and the byte-identical `article/published-original.md`, also embedded in this PDF as Supplement S1); the companion code (`companion/`, also as `downloads/utr-baselines.zip`, with the registered protocol in `PROTOCOL.md`); and the results archive (`downloads/utr-baselines-results.zip`) with metrics, per-fit receipts, predictions keyed by source index, verification outputs, device checks, the clean-environment rerun logs and the independent automated checks (its `MANIFEST.md` lists the contents).

Neither archive contains the source datasets, model weights or embedding caches; they are downloaded at runtime from pinned URLs and hash-checked. The results archive embeds only the

six example inserts of Table 3 and the constant 780-nt eGFP tail. The Hugging Face dataset card gives the processed data's licence as "unknown", so the source records are not redistributed; the embedded excerpts are retained for auditability, and their redistribution status is an open owner decision. Table 3 and Supplement S1 reproduce these six records exactly as the published article did and are subject to the same decision. GEO records are public; UTR-LM code and checkpoints are GPL-3.0 (its ESM files carry Meta's MIT header); mRNA Bench code is AGPL-3.0. The licence of the companion code itself is to be set at publication. The three reproduced literature figures are CC BY 4.0 and are attributed in their captions.

8 AI assistance, review and disclosure

The original article's drafts, editorial reviews and fact checks were automated (Claude Opus). The independent mechanical checks in the results archive were run by a separate automated orchestration session (Codex). The companion code was run on the machine described. This manuscript was drafted by an AI model (Claude Opus, `claude-opus-5-5`) from the archived evidence and reviewed by an independent AI reviewer instance of the same model; the review and its disposition are recorded in `evidence/paper-migration/review.md` and `review-disposition.md`. No human scientific review has taken place.

References

- [1] P. J. Sample et al. Human 5' UTR design and variant effect prediction from a massively parallel translation assay. *Nature Biotechnology*, 37:803–809, 2019. doi: 10.1038/s41587-019-0164-5. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC7100133/>. PMC7100133.
- [2] Y. Chu et al. A 5' UTR language model for decoding untranslated regions of mRNA and function predictions. *Nature Machine Intelligence*, 6:449–460, 2024. doi: 10.1038/s42256-024-00823-9. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC11155392/>. PMC11155392.
- [3] R. Shi, T. Dalal, P. Fradkin, et al. mRNA Bench. bioRxiv 2025.07.05.662870, CC BY 4.0, 2025. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC12265608/>. PMC12265608.
- [4] S. Castillo-Hair et al. Optimizing 5'UTRs for mRNA-delivered gene editing using deep learning. *Nature Communications*, 15:5284, 2024. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC11189900/>. PMC11189900; CC BY 4.0.
- [5] a96123155. UTR-LM code and checkpoints, commit b77b589 (GPL-3.0). GitHub, n.d. URL <https://github.com/a96123155/UTR-LM/tree/b77b589bf182eb9de6a1a5024fa09d44294d94fc>.
- [6] Y. Chu et al. Supplementary information for "a 5' UTR language model for decoding untranslated regions of mRNA and function predictions". *Nature Machine Intelligence*, 2024. URL https://media.springernature.com/original/springer-static/esm/art%3A10.1038%2Fs42256-024-00823-9/MediaObjects/42256_2024_823_MOESM1_ESM.pdf.
- [7] morrislab. mRNA Bench code and saved results, commit 74f96b8 (AGPL-3.0). GitHub, n.d.. URL <https://github.com/morrislab/mRNABench/tree/74f96b8e6ae9f41cc3cccf089d826a62d5604b8>. Saved results file: `experiments/overall_df.parquet` at the same commit.
- [8] A. Karollus, Ž. Avsec, and J. Gagneur. Predicting mean ribosome load for 5'UTR of any length using deep learning. *PLoS Computational Biology*, 17:e1008982, 2021. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC8136849/>. PMC8136849; CC BY 4.0.
- [9] NCBI Gene Expression Omnibus. Series GSE114002 and sample GSM3130443 (designed library), n.d. URL <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE114002>. Sample record: <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSM3130443>.
- [10] morrislab. mrl-sample dataset, revision ef67f7c, file `mrl-sample-designed.parquet`. Hugging Face, n.d.. URL <https://huggingface.co/datasets/morrislab/mrl-sample/tree/ef67f7cf8a999bb1c412ad6551aa7d9f901cbb95>.

- [11] rewire-benchmarks. Earlier composition and training-mean controls (local runs, 2026-09-20), commit ca73fa4. GitHub, n.d. URL <https://github.com/rewire-bio/rewire-benchmarks/blob/ca73fa47136d182f2d4ddb083d084712198fc0e2/research/local-runs-2026-09-20/README.md>.
- [12] MultiMolecule. utrln-mrl (excluded from this comparison). Hugging Face, n.d. URL <https://huggingface.co/multimolecule/utrln-mrl>.
- [13] bowang-lab. Orthrus README, commit 8a26aeb. GitHub, n.d. URL <https://github.com/bowang-lab/Orthrus/tree/8a26aebbc3883ab9d8b7efd773e1d913c0fbddaa>.
- [14] state-spaces. mamba-ssm at the release pinned by orthrus. GitHub, n.d. URL <https://github.com/state-spaces/mamba/tree/34076d664838588a3c97727b263478ab9f621a07>.
- [15] C. A. Field and A. H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B*, 69:369–390, 2007. doi: 10.1111/j.1467-9868.2007.00593.x. URL <https://doi.org/10.1111/j.1467-9868.2007.00593.x>.
- [16] PyTorch. Reproducibility notes (documentation 2.14), n.d. URL <https://docs.pytorch.org/docs/2.14/notes/randomness.html>.
- [17] uv documentation. uv, n.d. URL <https://docs.astral.sh/uv/>.

A Reproducing the historical run

The commands below are adapted from the original article (trailing comments moved to the text; one long command continued with a backslash; the tutorial notebook, installed with `uv sync --extra test --extra notebook`, is omitted). They were run on 30 September 2026 and were *not* re-executed for this manuscript. They need `uv` [17] and about 3 GB of free disk (0.85 GB environment, 31 MB inputs, 1.3 GB embedding cache). `uv sync` installs the locked environment (Python 3.11, NumPy 1.26.4, scikit-learn 1.5.2, torch 2.4.1); the first `pytest` reported 10 passed and 1 skipped. The smoke run uses every 20th record (5,001), two CNN epochs and seed 0; it is an installation check, not evidence. The full run took about 57 minutes on the Apple M4. Each `verify.py` call ends with ALL PASSED, and the second `pytest` (with `UTR_WORK` set) reported 11 passed. `make_figures.py` draws Figures 3–5 from `metrics.json`; `describe_failures.py` writes `failures.json`, the source of the family, composition, device-parity and per-seed timing numbers.

```
unzip utr-baselines.zip && cd utr-baselines
uv sync --extra test
uv run pytest -q

# Smoke run (installation check)
uv run python utr_baselines.py run-all --inputs runs/inputs \
    --work runs/smoke --smoke
uv run python verify.py --inputs runs/inputs --work runs/smoke

# Full run
uv run python utr_baselines.py run-all --inputs runs/inputs --work runs/full
uv run python verify.py --inputs runs/inputs --work runs/full
UTR_WORK=runs/full uv run pytest -q
uv run python make_figures.py --work runs/full --out runs/full/figures
uv run python describe_failures.py --work runs/full
```

The smoke run took 213s with CNN training on MPS, 199s in the clean rerun and 405s in a final check of the published archive; with `-cpu-only` and `OMP_NUM_THREADS=4` it took 190s in an independent check, and `verify.py` passed. A full `-cpu-only` run was not tested; at 151s per epoch it would take hours. The tutorial notebook runs the smoke configuration and took 230s using MPS. `run-all` runs `fetch` (15 downloads, each checked against a SHA-256 hash), `prepare` (constant leader and tail, exact GEO join, random-split hash 00e287b1...), `embed`, one `fit` process per method and split, and `evaluate`. A rerun well outside the seed ranges in Appendix Table 7 (grouped one-hot CNN 0.559 to 0.570) is a reason to investigate, not an expected tolerance.

Rebuilding this manuscript. `make paper-imported runs scripts/build_paper.py` with the system Python and local TeX Live. It verifies the archive and member digests, formats the tables in `paper/generated/`, converts the original flowchart SVG with `rsvg-convert`, and runs `pdflatex`, `BibTeX` and `pdflatex` twice. It performs no experiment, data download or environment creation. The receipt is `evidence/paper-migration/build-receipt.json`. The October 2026 repository audit added strict completeness, finite-value and artifact-provenance checks and automated regression tests to the maintained companion. These changes do not alter the archived measurements reported here. The original downloadable code remains preserved; the current maintained code is in `companion/`. This maintenance validation is not a new independent reproduction.

B The grouped split

Verbatim from `grouped_split` in the archived `downloads/utr-baselines.zip`; `GIANT_FRACTION` is 0.01 and the seed is 20260930. Components are built by linking records that share a GEO `mother` sequence or any identical 20-nt substring.

```

def grouped_split(comp: np.ndarray, seed: int = GROUPED_SEED) -> np.ndarray:
    n = len(comp)
    sizes = np.bincount(comp)
    split = np.full(n, "", dtype=object)
    giant = np.flatnonzero(sizes > GIANT_FRACTION * n)
    order = np.random.default_rng(seed).permutation(np.setdiff1d(np.arange(len(sizes)), giant))
    target = 0.15 * n
    filled = {"test": 0, "validation": 0}
    assign = {}
    for c in order:
        if filled["test"] < target:
            assign[c] = "test"; filled["test"] += sizes[c]
        elif filled["validation"] < target:
            assign[c] = "validation"; filled["validation"] += sizes[c]
        else:
            assign[c] = "train"
    for c in giant:
        assign[c] = "train"
    return np.array([assign[c] for c in comp], dtype=object)

```

C Supplementary tables

All values are formatted from the archived results without recomputation.

Table 6: Secondary metrics (MAE in MRL; Pearson and Spearman correlations, undefined for the constant predictor; enrichment = precision at 150 / test base rate; validation MSE; coverage), including the full-sequence composition replay. Source: archived `run/metrics.json`.

Split	Method	MAE	Pearson	Spearman	Enrichment	Val. MSE	Coverage
random	Composition, full sequence (replay)	1.101	0.437	0.495	3.45	1.872	15003/15003
random	Training mean	1.257	–	–	0.56	2.315	15003/15003
random	Composition	1.101	0.437	0.495	3.45	1.872	15003/15003
random	1–3-mer counts	0.832	0.695	0.704	5.63	1.219	15003/15003
random	Positional one-hot	1.055	0.502	0.544	3.80	1.735	15003/15003
random	uAUG/Kozak features	0.862	0.678	0.651	3.59	1.247	15003/15003
random	Cheap combined (reference)	0.668	0.801	0.797	6.12	0.823	15003/15003
random	CNN, one-hot (seed 0)	0.461	0.906	0.894	8.30	0.425	15003/15003
random	UTR-LM mean + ridge	0.820	0.707	0.703	4.85	1.170	15003/15003
random	UTR-LM per-position + ridge	0.731	0.777	0.765	5.42	0.936	15003/15003
random	UTR-LM + CNN head (seed 0)	0.453	0.911	0.893	6.96	0.405	15003/15003
grouped	Composition, full sequence (replay)	1.115	0.364	0.403	4.23	1.882	15004/15004
grouped	Training mean	1.192	–	–	0.72	2.258	15004/15004
grouped	Composition	1.115	0.364	0.403	4.23	1.882	15004/15004
grouped	1–3-mer counts	0.820	0.685	0.696	9.79	1.164	15004/15004
grouped	Positional one-hot	1.075	0.433	0.461	6.52	1.770	15004/15004
grouped	uAUG/Kozak features	0.834	0.675	0.645	5.56	1.241	15004/15004
grouped	Cheap combined (reference)	0.650	0.802	0.808	10.03	0.793	15004/15004
grouped	CNN, one-hot (seed 0)	0.551	0.862	0.855	10.39	0.542	15004/15004
grouped	UTR-LM mean + ridge	0.817	0.691	0.693	6.28	1.112	15004/15004
grouped	UTR-LM per-position + ridge	0.775	0.729	0.731	6.89	1.032	15004/15004
grouped	UTR-LM + CNN head (seed 0)	0.546	0.860	0.852	10.27	0.534	15004/15004

Table 5: All 22 registered paired comparisons. Positive favours the named method over the reference.
Source: archived `run/metrics.json, splits.<split>.comparisons`.

Split	Method	Reference	MSE reduction [95% CI]	Verdict	P@150 gain [95% CI]	Verdict
random	Composition	cheap combined	-1.065 [-1.171, -0.963]	worse	-0.253 [-0.336, -0.141]	worse
random	1-3-mer counts	cheap combined	-0.375 [-0.406, -0.348]	worse	-0.047 [-0.119, +0.108]	no supported difference
random	Positional one-hot	cheap combined	-0.920 [-1.002, -0.836]	worse	-0.220 [-0.336, -0.133]	worse
random	uAUG/Kozak features	cheap combined	-0.431 [-0.467, -0.398]	worse	-0.240 [-0.373, -0.039]	worse
random	CNN	cheap combined	+0.422 [+0.363, +0.488]	adds practical value	+0.207 [+0.124, +0.292]	adds practical value
random	UTR-LM mean + ridge	cheap combined	-0.335 [-0.371, -0.305]	worse	-0.120 [-0.203, -0.015]	worse
random	UTR-LM per-position + ridge	cheap combined	-0.090 [-0.144, -0.033]	worse	-0.067 [-0.141, +0.056]	no supported difference
random	UTR-LM + CNN	cheap combined	+0.443 [+0.388, +0.502]	adds practical value	+0.080 [+0.000, +0.208]	no supported difference
random	UTR-LM mean + ridge	CNN	-0.757 [-0.812, -0.698]	worse	-0.327 [-0.392, -0.215]	worse
random	UTR-LM per-position + ridge	CNN	-0.512 [-0.536, -0.484]	worse	-0.273 [-0.343, -0.155]	worse
random	UTR-LM + CNN	CNN	+0.021 [+0.009, +0.034]	below practical threshold	-0.127 [-0.179, -0.023]	worse
grouped	Composition	cheap combined	-1.112 [-1.199, -1.035]	worse	-0.320 [-0.453, -0.195]	worse
grouped	1-3-mer counts	cheap combined	-0.386 [-0.421, -0.349]	worse	-0.013 [-0.107, +0.085]	no supported difference
grouped	Positional one-hot	cheap combined	-0.992 [-1.055, -0.932]	worse	-0.193 [-0.338, -0.075]	worse
grouped	uAUG/Kozak features	cheap combined	-0.417 [-0.460, -0.372]	worse	-0.247 [-0.375, -0.128]	worse
grouped	CNN	cheap combined	+0.216 [+0.176, +0.252]	adds practical value	+0.020 [-0.088, +0.132]	no supported difference
grouped	UTR-LM mean + ridge	cheap combined	-0.363 [-0.400, -0.327]	worse	-0.207 [-0.335, -0.073]	worse
grouped	UTR-LM per-position + ridge	cheap combined	-0.244 [-0.272, -0.216]	worse	-0.173 [-0.278, -0.047]	worse
grouped	UTR-LM + CNN	cheap combined	+0.206 [+0.173, +0.237]	adds practical value	+0.013 [-0.087, +0.149]	no supported difference
grouped	UTR-LM mean + ridge	CNN	-0.579 [-0.624, -0.531]	worse	-0.227 [-0.357, -0.107]	worse
grouped	UTR-LM per-position + ridge	CNN	-0.460 [-0.501, -0.418]	worse	-0.193 [-0.302, -0.062]	worse
grouped	UTR-LM + CNN	CNN	-0.010 [-0.034, +0.016]	no supported difference	-0.007 [-0.103, +0.122]	no supported difference

Table 7: Training-seed variability of the CNN heads: test MSE / precision at 150 for each seed, and seconds per seed. Paired intervals use seed 0, fixed in advance. Source: archived `run/metrics.json` (`per_seed`) and `run/failures.json` (`seconds_this_seed`).

Split	Head	Seed 0	Seed 1	Seed 2	Seconds per seed
random	CNN	0.427 / 0.787	0.439 / 0.747	0.413 / 0.713	205, 174, 209
random	UTR-LM + CNN	0.407 / 0.660	0.412 / 0.713	0.410 / 0.687	269, 313, 294
grouped	CNN	0.560 / 0.573	0.559 / 0.533	0.570 / 0.580	250, 279, 204
grouped	UTR-LM + CNN	0.570 / 0.567	0.552 / 0.560	0.538 / 0.620	263, 249, 264

Table 8: Test MSE by read-depth quartile for the seed-0 CNN and the cheap-combined reference (quartile edges rounded to whole reads; point estimates, no intervals). Source: archived `run/metrics.json` and `run/failures.json`.

Split	Quartile	Reads	CNN MSE	Reference MSE
random	Q1 (fewest reads)	100–318	0.626	1.156
random	Q2	318–624	0.438	0.872
random	Q3	624–1,154	0.373	0.700
random	Q4 (most reads)	1,154–22,646	0.272	0.666
grouped	Q1 (fewest reads)	100–315	0.785	1.024
grouped	Q2	315–616	0.540	0.777
grouped	Q3	616–1,129	0.495	0.697
grouped	Q4 (most reads)	1,129–22,646	0.419	0.606

Table 9: Random and grouped test MSE by sub-library for the reference, the seed-0 CNN and the UTR-LM CNN head (point estimates; some grouped sub-libraries have only five test families). Source: archived `run/metrics.json`, `per_library_mse`.

Sub-library	Random split			Grouped split		
	Cheap comb.	CNN	UTR-LM + CNN	Cheap comb.	CNN	UTR-LM + CNN
egfp_controls	0.896	0.498	0.424	0.935	0.567	0.587
egfp_poor_performers	1.496	0.883	0.689	1.443	1.154	1.095
human_utrs	0.778	0.447	0.438	0.723	0.505	0.513
mcherry	0.710	0.373	0.352	0.658	0.460	0.490
sergii_utrs	0.991	0.544	0.499	1.055	0.687	0.749
snv	0.696	0.312	0.307	0.708	0.576	0.636
step_random_to_best_allow_uatg	0.855	0.399	0.437	0.657	0.498	0.815
step_random_to_best_no_uatgs	0.902	0.192	0.181	–	–	–
step_worst_to_best_allow_uatg	0.723	0.314	0.253	0.405	1.134	0.522
step_worst_to_best_no_uatg	0.852	0.262	0.270	0.554	0.527	0.621
target_allow_uaug_allow_stop	1.146	0.499	0.455	0.804	0.494	0.513
target_no_uaug_allow_stop	1.140	0.462	0.463	0.892	0.503	0.446
target_no_uaug_no_stop	0.790	0.334	0.364	0.572	0.498	0.435

Table 10: Library composition: records per sub-library, their train/validation/test allocation and the number of test-set families (components with at least one test record in that sub-library) under each split; families can span sub-libraries, so these columns do not sum to the 9,640 / 6,002 test components. Source: archived `run/prepare.json` and `run/failures.json`.

Sub-library	Records	Random split		Grouped split	
		Train/val/test	Test families	Train/val/test	Test families
egfp_controls	5,971	4,219/915/837	744	3,917/964/1,090	585
egfp_poor_performers	3,257	2,271/503/483	434	2,113/521/623	314
human_utrs	41,862	29,349/6,193/6,320	5,133	27,125/7,442/7,295	3,125
mcherry	5,963	4,232/852/879	796	3,872/1,041/1,050	554
sergii_utrs	4,882	3,401/745/736	675	3,198/784/900	497
snv	8,738	6,089/1,336/1,313	644	5,663/1,570/1,505	175
step_random_to_best_allow_uatg	2,427	1,713/374/340	17	2,187/59/181	5
step_random_to_best_no_uatgs	2,499	1,732/388/379	5	2,479/20/0	0
step_worst_to_best_allow_uatg	4,847	3,394/709/744	38	4,070/567/210	5
step_worst_to_best_no_uatg	4,403	3,069/675/659	45	3,552/351/500	9
target_allow_uaug_allow_stop	5,020	3,502/765/753	547	3,583/750/687	349
target_no_uaug_allow_stop	4,848	3,384/738/726	362	3,901/444/503	262
target_no_uaug_no_stop	5,300	3,656/810/834	376	4,336/504/460	230

Table 11: CNN device parity: validation MSE computed on the training device and on CPU after training, for all 12 CNN fits. Agreement of one metric, not a general device-parity guarantee. Source: archived `run/failures.json`, `cnn_device_parity`.

Split	Head	Seed	Device	Val. MSE (device)	Val. MSE (CPU)	$ \Delta $
random	CNN	0	mps	0.42460428	0.42460425	3.3e-08
random	CNN	1	mps	0.42664074	0.42664072	1.8e-08
random	CNN	2	mps	0.40005119	0.40005118	1.6e-08
random	UTR-LM + CNN	0	mps	0.40532692	0.40532693	3.4e-09
random	UTR-LM + CNN	1	mps	0.41623632	0.41623633	9.6e-09
random	UTR-LM + CNN	2	mps	0.39967982	0.39967982	9.0e-11
grouped	CNN	0	mps	0.54151269	0.54151267	1.5e-08
grouped	CNN	1	mps	0.54634050	0.54634048	2.0e-08
grouped	CNN	2	mps	0.55582367	0.55582366	1.4e-08
grouped	UTR-LM + CNN	0	mps	0.53374345	0.53374343	2.5e-08
grouped	UTR-LM + CNN	1	mps	0.53037857	0.53037858	2.1e-09
grouped	UTR-LM + CNN	2	mps	0.52518994	0.52518994	5.3e-09

Table 12: Per-step process wall time in the full run (seconds), from the archived `run/run-log.json`. Ridge rows include process start-up and differ slightly from the in-process fit times in Table 1; CNN rows cover three seeds, whereas Table 1 gives seed 0 (in-process three-seed totals are in Section 4.3). The steps sum to 3,415.2 s; the shell-measured whole run was 3,417 s.

Step	Seconds
fetch	0.5
prepare	3.6
embed	137.5
random: Training mean	0.7
random: Composition, full sequence (replay)	2.2
random: Composition	0.9
random: 1–3-mer counts	13.8
random: Positional one-hot	1.7
random: uAUG/Kozak features	2.0
random: Cheap combined (reference)	15.3
random: CNN, one-hot (3 seeds)	588.5
random: UTR-LM mean + ridge	1.3
random: UTR-LM per-position + ridge	106.8
random: UTR-LM + CNN head (3 seeds)	877.4
grouped: Training mean	0.9
grouped: Composition, full sequence (replay)	2.3
grouped: Composition	0.6
grouped: 1–3-mer counts	13.5
grouped: Positional one-hot	1.0
grouped: uAUG/Kozak features	1.3
grouped: Cheap combined (reference)	21.3
grouped: CNN, one-hot (3 seeds)	734.6
grouped: UTR-LM mean + ridge	1.1
grouped: UTR-LM per-position + ridge	90.4
grouped: UTR-LM + CNN head (3 seeds)	778.4
evaluate	17.5

Table 13: Independent mechanical checks by `verify.py` (no shared code with the main script); check names are verbatim from the archive, where “historical” is the random split. Source: archived `run/verify.json`.

Check	Result	Detail
historical split hash	passed	00e287b18ba4e90c...
historical labels match records	passed	
grouped components reproduced by separate BFS	passed	33575 components
no component spans grouped splits	passed	max splits per component 1
replay reproduces published historical MSE	passed	1.9144397158415434 vs 1.914439715839851
metrics recomputed independently	passed	max abs difference 0
grouped comp_utr refit with scikit-learn	passed	max abs prediction difference 1.38e-07
historical comp_utr refit with scikit-learn	passed	max abs prediction difference 1.43e-07

Table 14: Clean-environment rerun on the same machine: original and rerun test MSE and the maximum absolute prediction difference. Source: `compare-with-original.json` in the archived `receipts/clean-room/`.

Split	Method	Original MSE	Rerun MSE	Max $ \Delta $
random	uAUG/Kozak features	1.279831	1.279831	0
random	Cheap combined (reference)	0.849202	0.849202	0
random	Composition, full sequence (replay)	1.914440	1.914440	0
random	Composition	1.914438	1.914438	0
random	1–3-mer counts	1.224664	1.224664	0
random	Positional one-hot	1.768726	1.768726	0
random	Training mean	2.365529	2.365529	0
random	UTR-LM mean + ridge	1.184381	1.184381	0
random	UTR-LM per-position + ridge	0.939030	0.939030	0
grouped	uAUG/Kozak features	1.192937	1.192937	0
grouped	Cheap combined (reference)	0.775964	0.775964	0
grouped	CNN, one-hot	0.559972	0.559972	0
grouped	Composition, full sequence (replay)	1.888194	1.888194	0
grouped	Composition	1.888203	1.888203	0
grouped	1–3-mer counts	1.161589	1.161589	0
grouped	Positional one-hot	1.767481	1.767481	0
grouped	Training mean	2.224258	2.224258	0
grouped	UTR-LM mean + ridge	1.138743	1.138743	0
grouped	UTR-LM per-position + ridge	1.020193	1.020193	0

D Selection flowchart

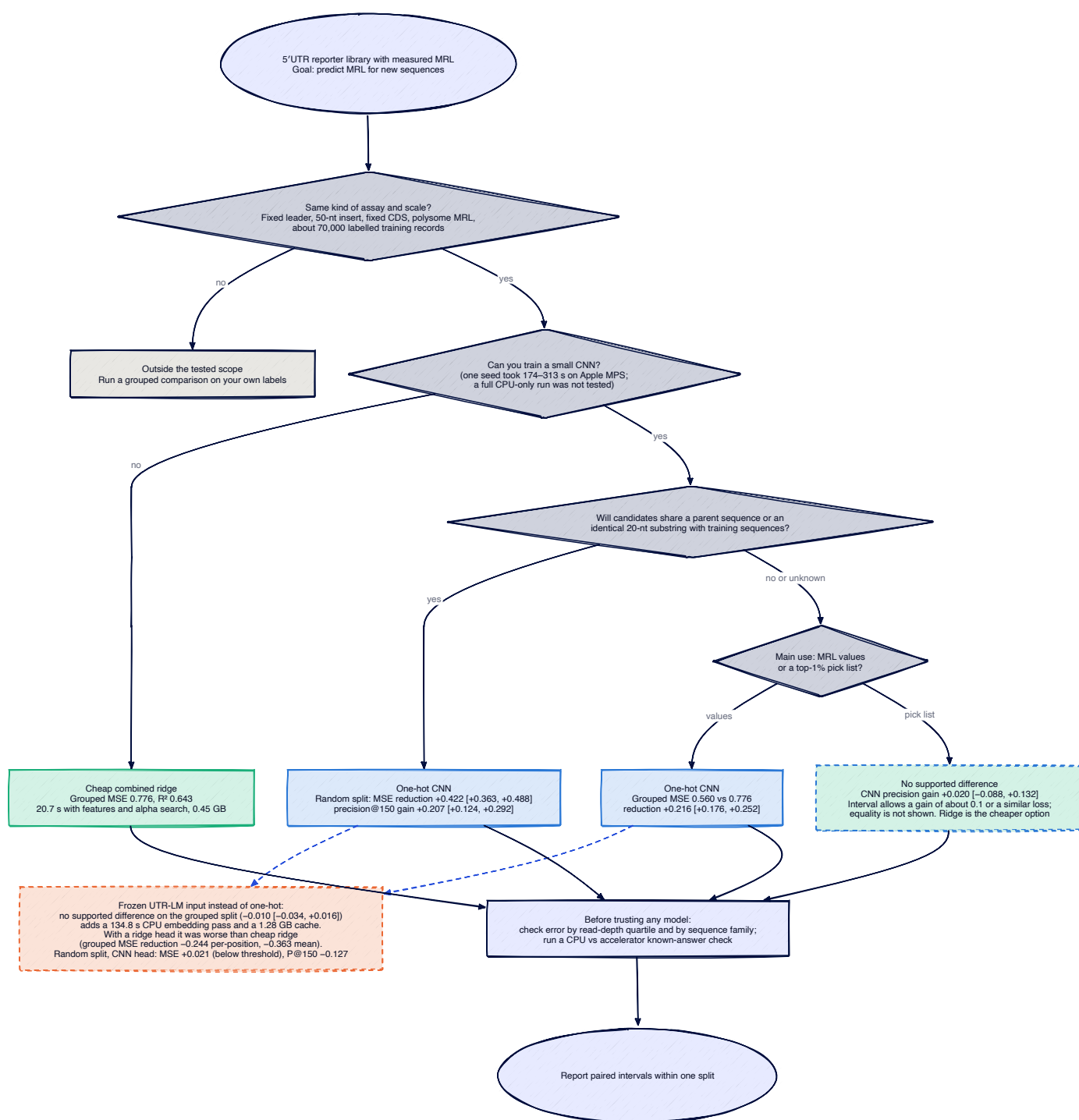


Figure 7: Selection flowchart (original Figure 6, converted from `article/assets/06-selection-flowchart.svg`; D2 source in `06-selection-flowchart.d2`), built from the original Tables 1–3 (here Tables 4, 1 and 2). First check the assay and label scale are like this one; if a CNN cannot be trained use cheap combined ridge; if candidates are relatives of training sequences, or MRL values are needed, use the one-hot CNN; for a top-1% pick list there is no supported difference and ridge is cheaper; frozen UTR-LM input is a side note with no supported difference; all paths end with checks by read depth and family. The vector figure can be zoomed; Table 4 gives the same routes in readable form.

E Provenance, supplement and evidence mapping

Supplement S1 (original article). The full original article, *Choosing a Model for 5' UTR Translation Experiments: Cheap Features, a Small CNN or Frozen UTR-LM* (30 September 2026), is retained unchanged in the repository as `article/published-original.md` (byte-identical to the published source) and is embedded in this PDF as the attachment `S1-published-original.md`. The repository copy `article/original.md` differs only in local links. Every scientific result it reports appears in this manuscript; the section-by-section mapping is in `evidence/paper-migration/content-coverage.md`.

Evidence ledger. `evidence/paper-migration/claims-ledger.json` links each substantive claim to the historical artifact (archive member or article text), a pointer inside it and its SHA-256. The results archive was read in memory without extraction; its member digests are recorded in `evidence/migration-audit.json`. The table generator (`scripts/paper_extract.py`) stops the build if any generated value that also appears in the original article differs from it: every cell of Tables 1 and 2 (including verdicts), the read-depth and trajectory-family values, and the dependence, base-rate and resource numbers in the text. The recorded cross-check found no difference. Values that the original did not print (the appendix tables) are formatted directly from the digest-checked archive members.

Citation note. Literature claims were checked for the original article in September 2026 and were not re-retrieved during migration. DOIs for Sample et al., Chu et al. and mRNABench are those printed in the original companion's `NOTICE.md`.